1. General Information

- Title: Map guessing game
- Creator: Minh Tran
- Student number: 101555658
- Study program: Digital Systems and Design
- Study year: third year
- Date: 20/2/2026

2. General Description and Difficulty Level

- This project is an interactive desktop application that teaches world geography through exploration and competitive gaming. The application features an interactive map where users can discover country details, a "Guess by Shape" mode with visual hints, a "Flag Master" mode, and a high-speed "Time Attack" challenge.
- Unique feature:
    - A "Dynamic Heatmap" progress system. As the user correctly identifies countries, the world map is permanently updated with a color gradient representing the user's mastery level of different continents.
    - A counter to track the distance traveled after correctly guessing a country (distance between capitals). Using Haversine formula to calculate distance on a sphere.
- Difficulty level: I'm aiming for the hard level.

3. Use Case and User Interface

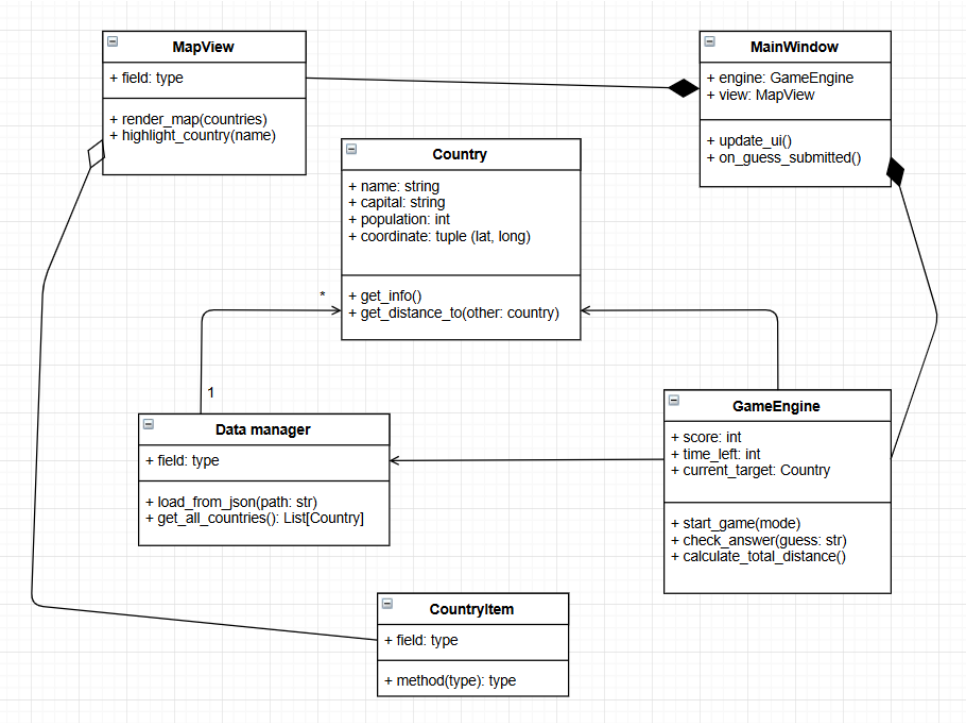
- UI description: the main window consists of a large QGraphicsView displaying a world map. On the right, a side panel shows statistics (score, progress). At the top, a navigation bar allows switching between "Explore," "Shape Quiz," "Flag Quiz," and "Time Attack."
- Use case:
    - User action: click the "Time Attack" and "Start"
        - Behind the scene: the GameEngine initializes a Qtimer (60s) and selects a random country
    - User action: sees a country outline (shape quiz) and types the name into a QlineEdit
        - Behind the scene: GameEngine compares input to the Country object's name. If correct, it increments the score and triggers a

"Success" signal to the MapView to highlight the country in green. Also provide some information about the country.

- User action: if feeling stuck, clicks “Hint”
  - Behind the scene: the program reveals some hints (like the capital, etc)

4. Structure of the Program

- The program follows a Model-View-Controller (MVC) inspired architecture to separate data from the GUI.
  - Country (model): Stores attributes (name, capital, population, flag_path, coordinates, SVG path).
  - DataManager: Read JSON files and turn that into a list of Country objects.
  - GameEngine: Manages game logic, scoring, timers, and hint generation.
  - MainWindow and MapView (UI): This is PyQt side:
    - MainWindow hold buttons and labels
    - MapView (extending QgraphicsView) draws the CountryItem shapes
  - CountryItem: A custom QgraphicsPathItem representing a single country on the map. It knows when it is clicked.



5. Data Structures

- Custom objects (country class): the core data structure is a collection of Country objects, which encapsulate all related attributes like name, capital, and geographic coordinates.
- Dictionary (Hash map): A dictionary will be used to map country names (keys) to their corresponding Country objects (value). This allows for O(1) constant time lookup (when a user submit the guess, the program needs to verify it instantly.
    - For visual heat map, use another dictionary to track "mastery level".
- Lists (dynamic arrays): a list of all available countries, used for the random selection process in game modes. (use lists for easy shuffling and indexed access)
- Set: to track "Already Guessed" countries to ensure no duplicates in a single session. Works by storing IDs of countries which are chosen before. (use sets because they inherently prevent duplicates).
- QpainterPath: the container for painting operations. It stores the geometric outlines of countries as a sequence of mathematical lines and curves.

6. Files and File Formats

- countries.json: the primary data file. A text file containing all country data (name, capital, etc)
- map_data.svg: containing the coordinates for country outlines. Used in shape guess game mode.
- /flags/ folder: a directory of files named by country code (fi for Finland). Contain image files for the flags game mode.
- userdata.json: stores the user's progress and high scores

7. Algorithms

- String matching: use a basic fuzzy matching approach
    - For "time attack" mode
    - Use Levenshtein distance (set a threshold if the number of changes needed less than a number)
- Randomization: ensure quiz order is random and unique every time (Fisher-Yates shuffle algorithm)
- Coordinate mapping: use Mercator Projection

- For x - axis: linear transform
  x = (lon + 180) x (map_width / 360)
- For y – axis:
  y = ln(tan(pi/4 + lat_radians/2))
  then scale this y to fit the window height.

8. Testing Plan

- Unit testing:
  - Test GameEngine.check_guess()
  - Test DataManager.load_data() to prevent the program crash if JSON failed
- UI and system testing:
  - Verify clicking a country open the correct info panel
  - Test Qtime stops exactly a zero
- Boundary test:
  - Test countries with special name (2 word name, contain special character, etc)
- Logic test:
  - Test Haversine function
  - Test fuzzy input answer
- Function test:
  - Test Heatmap color
  - Flag-to-country are connected correctly

9. Libraries and Tools

- PyQt6: for graphical UI and event handling
- json: for data storage
- random: for random quiz

10. Schedule

- Research and data preparation (5h): JSON and SVG data gathered
- Code architecture (8h): Data classes and basic QGraphicsView map rendering.
- Checkpoint 3 (10h): Working map when clicking a country show its name
- Game logic (10h): Implementation of Shape Quiz and Flag Quiz modes
- Advanced features (8h): Timer challenge, scoring system, and hints.
- Checkpoint 6 (6h): Unit tests for game logic and data loading completed

- Polishing and UI (5h): Final visual tweaks and bug fixing.

11. References

- PyQt6: https://www.riverbankcomputing.com/static/Docs/PyQt6/
- RestCountries API (for data source): https://restcountries.com/
- Natural earth (for map vector data): https://www.naturalearthdata.com/
- Python 3.x standard library documentation